# claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf_b531eaef-39c\agent-a0c16c025d7a999d5.jsonl`
- SHA-256: `b5cfb7f8ea60c204ac3328740d52ce5d2f913efa4a2bffc73ea011b2f272c72c`
- Source modified: `2026-06-19T16:57:34+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_b531eaef-39c`
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`

## Transcript

#### 1. user (2026-06-19T16:57:07.027Z)

You are verifying claims for a design doc; the doc must only assert what the code supports. Verify against the code in backend/pipeline/detectors/wasb_infer.py: (a) the WASB detector returns a LIST of candidate detections PER FRAME (each {xy,score,scale}); (b) det_by_fid accumulates ALL candidates; (c) the single-target tracker (build_tracker) collapses the candidate list to ONE (visi,x,y) per frame via tracker.update. Cite exact line numbers. State VERIFIED/REFUTED/PARTIAL per sub-claim with the evidence line.

Read the actual files (do not guess). Return the structured verdict.

#### 2. assistant (2026-06-19T16:57:09.853Z)

I'll verify these claims against the actual code. Let me read the file first.

#### 3. user (2026-06-19T16:57:10.813Z)

```
1       """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3       WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4       own building blocks (ImageDataset transform + detector + online tracker) to run on
5       any folder of extracted frames and emit a trajectory CSV in the exact shape the
6       indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8           Frame,Visibility,X,Y
9
10      It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11      imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12      (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14      Resumable across reboots
15      ------------------------
16      The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17      the slow, expensive part; the CPU tracker pass that turns detections into a
18      trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19      resume mid-stream). We exploit that split:
20
21        * The detector pass writes its per-window raw outputs incrementally to
22          ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23          records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24          most ``--flush-every`` windows of GPU work instead of the whole pass.
25        * On restart we replay the cached windows, skip the DataLoader ahead to the first
26          un-cached window, and continue. Once every window is cached, the detector pass
27          is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28          the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29          not the GPU hour.
```

```
30
31      The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32      to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33      weights, sport, window size, or frame count change.
34
35      Usage (in WSL, from ~/models/WASB-SBDT/src):
36          python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
37              --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
every 1000] [--fresh]
38      """
39      import os
40      import os.path as osp
41      import csv
42      import glob
43      import json
44      import argparse
45      import logging
46      from collections import defaultdict
47      from contextlib import contextmanager
48
49      logger = logging.getLogger(__name__)
50
51      CACHE_VERSION = 1
52      _DET_FILE = "det_raw.jsonl"
53      _MANIFEST_FILE = "manifest.json"
54
55
56      def _build_cfg(weights: str, sport: str):
57          """Compose the WASB eval config via Hydra, overriding model/weights/device."""
58          from hydra import compose, initialize
59          with initialize(config_path="configs", version_base=None):
60              cfg = compose(config_name="eval", overrides=[
61                  f"dataset={sport}",
62                  "model=wasb",
63                  f"detector.model_path={weights}",
64                  "runner.device=cuda",
65                  "runner.gpus=[0]",      # this box has a single GPU; default config assumes
more
66              ])
67          return cfg
68
69
70      def _frame_id(path: str) -> int:
71          """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
72          stem = osp.splitext(osp.basename(path))[0]
73          digits = "".join(ch for ch in stem if ch.isdigit())
74          return int(digits) if digits else -1
75
76
77      # ---------------------------------------------------------------------------- #
78      # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
79      # ---------------------------------------------------------------------------- #
80      def _cache_paths(cache_dir: str):
81          return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
82
83
84      def default_cache_dir(out_csv: str) -> str:
85          """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
86          d = osp.dirname(osp.abspath(out_csv))
87          stem = osp.splitext(osp.basename(out_csv))[0]
88          return osp.join(d, stem + "_wasbcache")
89
90
```

```
    def _atomic_write_text(path: str, text: str) -> None:
        """Write then rename so a crash mid-write can't leave a half-written file."""
        tmp = path + ".tmp"
        with open(tmp, "w") as f:
            f.write(text)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, path)


    def _det_plain(det) -> list:
        """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
        xy = det["xy"]
        return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]


    def _dets_to_tracker(plain_list):
        """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
        the tracker does vector math / np.linalg.norm on it)."""
        import numpy as np
        return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
                for p in plain_list]


    def write_manifest(cache_dir: str, manifest: dict) -> None:
        _, mpath = _cache_paths(cache_dir)
        _atomic_write_text(mpath, json.dumps(manifest, indent=2))


    def read_manifest(cache_dir: str):
        _, mpath = _cache_paths(cache_dir)
        if not osp.exists(mpath):
            return None
        try:
            with open(mpath) as f:
                return json.load(f)
        except (json.JSONDecodeError, OSError):
            return None


    def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
                            frames_in: int, frames_total: int) -> bool:
        """A cache is reusable only if the run that produced it matches this run's
inputs."""
        if not manifest or manifest.get("version") != CACHE_VERSION:
            return False
        return (
            manifest.get("frames_dir") == osp.abspath(frames_dir)
            and manifest.get("weights") == weights
            and manifest.get("sport") == sport
            and manifest.get("frames_in") == frames_in
            and manifest.get("frames_total") == frames_total
        )


    def reset_cache(cache_dir: str) -> None:
        """Drop any existing cache so the next run starts clean."""
        det_jsonl, mpath = _cache_paths(cache_dir)
        for p in (det_jsonl, mpath):
            if osp.exists(p):
                os.remove(p)


```

```
    def load_and_clean_cache(cache_dir: str):
        """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
        rebuild the per-frame detection accumulation, and rewrite the file to exactly that
        prefix so a trailing half-written line from a crash can't corrupt future appends.

        Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
        [x, y, score, scale] candidates (accumulated across every window the frame is in,
        in window order ? identical to a non-resumed run).
        """
        det_jsonl, _ = _cache_paths(cache_dir)
        det_by_fid = defaultdict(list)
        valid: list = []
        if osp.exists(det_jsonl):
            with open(det_jsonl, "r") as f:
                for line in f:
                    s = line.strip()
                    if not s:
                        continue
                    try:
                        obj = json.loads(s)
                    except json.JSONDecodeError:
                        break                       # truncated trailing line (crash mid-
flush)
                    if obj.get("w") != len(valid):  # non-contiguous -> stop at the gap
                        break
                    valid.append(s)
                    for fid, plain in obj["f"]:
                        det_by_fid[int(fid)].extend([list(p) for p in plain])
            # Guarantee a clean append boundary by rewriting only the good prefix.
            _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
        return len(valid), det_by_fid


    def _append_windows(fh, objs) -> None:
        """Append window records as JSONL and durably flush (bounded loss on crash)."""
        for o in objs:
            fh.write(json.dumps(o, separators=(",", ":")) + "\n")
        fh.flush()
        os.fsync(fh.fileno())


    def _atomic_write_trajectory(out_csv: str, rows) -> None:
        os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
        tmp = out_csv + ".tmp"
        with open(tmp, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["Frame", "Visibility", "X", "Y"])
            w.writerows(rows)
            f.flush()
            os.fsync(f.fileno())
        os.replace(tmp, out_csv)


    # ---------------------------------------------------------------------------- #
    # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
    # ---------------------------------------------------------------------------- #
    _STREAM_FRAME_FMT = "{:08d}.png"


    def synthetic_frame_path(index: int) -> str:
        """Stable, index-encoding frame name used by the streaming path.

        It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
        downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
        frames came from disk or from the in-memory stream. The streaming image loader
```

```
        inverts this name back to an index to fetch the decoded frame.
        """
        return _STREAM_FRAME_FMT.format(int(index))


    def build_windows(frames: list, frames_in: int):
        """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
        + checkpoint). Pure list math, identical for the disk and streaming paths.

        Returns a list of windows, each a list of ``frames_in`` consecutive paths
        (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
        caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
        ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
        """
        return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]


    class SequentialFrameStore:
        """In-memory sliding buffer over a forward-only frame reader, sized for the
        detector's sliding-window access pattern (peak O(window) frames, not O(video)).

        The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
        ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
        in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
        start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
        boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
        far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
        earliest index any not-yet-finished window can still ask for) and decode each source
        frame exactly once via the forward-only ``reader(index)``.
        """

        def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
            self._reader = reader
            self._total = int(total)
            self._window = max(1, int(window))
            self._buf: dict = {}
            # On a resumed run the detector's first needed frame is `start_index`; begin
            # pulling there so the reader can skip (seek past) the already-cached prefix
            # instead of decoding 0..start_index-1 just to throw them away.
            self._next = int(start_index)       # next index not yet pulled from the reader
            self._max_seen = int(start_index) - 1
            self._floor = int(start_index)      # lowest index we still keep (never evict
below)

        def get(self, index: int):
            index = int(index)
            if index < self._floor:
                raise KeyError(
                    f"frame {index} already evicted (below floor {self._floor}); the access
"
                    f"pattern must stay within a window of the highest frame seen")
            if index >= self._total:
                raise KeyError(f"frame {index} out of range (total {self._total})")
            # Pull forward from the reader until the requested index is buffered.
            while self._next <= index:
                self._buf[self._next] = self._reader(self._next)
                self._next += 1
            if index > self._max_seen:
                self._max_seen = index
            # Evict frames older than the earliest a still-running window could re-request:
            # once we've seen index `m`, no future window starts before `m - (window-1)`.
            new_floor = max(self._floor, self._max_seen - (self._window - 1))
            for k in range(self._floor, new_floor):
                self._buf.pop(k, None)
            self._floor = new_floor
```

```
280             return self._buf[index]
281
282
283     def _index_from_synthetic_path(path: str) -> int:
284         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
285
286         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
287         in lockstep with the on-disk naming scheme.
288         """
289         return _frame_id(path)
290
291
292     def _bgr_to_pil_rgb(frame_bgr):
293         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
294         PIL RGB image WASB's disk path produces.
295
296         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
297         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
298         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
299         frame indistinguishable from the on-disk one to everything downstream.
300         """
301         import cv2
302         from PIL import Image
303         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
304
305
306     def _make_streamed_read_image(store, real_read_image):
307         """Build the ``read_image`` replacement used during a streaming detector pass.
308
309         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
310         matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
311         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
312         without the WSL-only ``dataloaders`` package.
313         """
314         def _read_image(path, *args, **kwargs):
315             idx = _index_from_synthetic_path(path)
316             if idx < 0:
317                 return real_read_image(path, *args, **kwargs)
318             return _bgr_to_pil_rgb(store.get(idx))
319         return _read_image
320
321
322     @contextmanager
323     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
int = 0):
324         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
325         frames during the detector pass, byte-for-byte as it would over real PNGs.
326
327         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
328         ``cv2.imread``. ``read_image`` (``utils.utils``) does
``Image.open(path).convert('RGB')``,
329         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
330         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
331         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
332         ``osp.exists`` guard is sidestepped for synthetic stream paths.
```

```
333
334         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
335         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
336         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
337         reference).
338
339         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
340         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
341         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
342         original reader is restored on exit.
343         """
344         from dataloaders import dataset_loader as _dl
345         store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
346         real_read_image = _dl.read_image
347         # Rebind read_image in the consuming module for the duration of the detector pass;
348         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
349         _dl.read_image = _make_streamed_read_image(store, real_read_image)  # type:
ignore[assignment]
350         try:
351             yield store
352         finally:
353             _dl.read_image = real_read_image  # type: ignore[assignment]
354
355
356     # ------------------------------------------------------------------------ #
357     # Inference
358     # ------------------------------------------------------------------------ #
359     def run(frames, weights: str, out_csv: str, sport: str = "badminton",
360             limit: int = 0, batch_size: int = 8, num_workers: int = 4,
361             log_every_batches: int = 50, cache_dir: "str | None" = None,
362             flush_every: int = 1000, fresh: bool = False,
363             frame_loader=None, source_key: "str | None" = None) -> str:
364         """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
365
366         Two equivalent ways to supply pixels (output is bit-identical between them):
367
368         * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
369           globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
``cv2.imread``.
370         * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
371           frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
372           pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
373           reader, which returns a PIL RGB image) over the DataLoader pass so every other
line of
374           ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
identical
375           to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
376           ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
``num_workers=0``
377           (the in-process shim + buffer can't cross worker processes).
378
379         ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
380         frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
381         after reboot still matches.
382         """
383         import time
384         import torch
```

```
        from torch.utils.data import DataLoader
        from detectors import build_detector
        from trackers import build_tracker
        from dataloaders import build_img_transforms, build_seq_transforms
        from dataloaders.dataset_loader import ImageDataset
        from utils import Center

        streaming = frame_loader is not None

        cfg = _build_cfg(weights, sport)
        frames_in = int(cfg["model"]["frames_in"])
        input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
        output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))

        if streaming:
            # `frames` is already the ordered list of (synthetic) frame paths.
            if not isinstance(frames, (list, tuple)):
                raise SystemExit("streaming mode requires `frames` to be a list of frame
paths")
            frames = list(frames)
            frames_dir = source_key or "stream"
        else:
            frames_dir = frames
            frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) +
glob.glob(osp.join(frames_dir, "*.jpg")))
        if limit:
            frames = frames[:limit]
        if len(frames) < frames_in:
            where = frames_dir if not streaming else "video stream"
            raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")

        # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
        samples = []
        for window in build_windows(frames, frames_in):
            annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
            samples.append({"frames": window, "annos": annos})
        windows_total = len(samples)

        # ---- cache setup / resume decision ---------------------------------- #
        if cache_dir is None:
            cache_dir = default_cache_dir(out_csv)
        os.makedirs(cache_dir, exist_ok=True)
        det_jsonl, _ = _cache_paths(cache_dir)
        cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
        manifest = None if fresh else read_manifest(cache_dir)
        resume = manifest_compatible(
            manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
            frames_in=frames_in, frames_total=len(frames),
        )
        if resume:
            windows_done, det_by_fid = load_and_clean_cache(cache_dir)
            logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
        else:
            if manifest is not None:
                logger.info("cache incompatible with this run -> starting fresh")
            reset_cache(cache_dir)
            windows_done, det_by_fid = 0, defaultdict(list)

        base_manifest = {
            "version": CACHE_VERSION,
            "frames_dir": cache_key,
            "weights": weights,
```

```
            "sport": sport,
            "frames_in": frames_in,
            "frames_total": len(frames),
            "windows_total": windows_total,
            "windows_done": windows_done,
        }
        write_manifest(cache_dir, base_manifest)

        # ---- detector pass over the un-cached windows ----------------------- #
        remaining = samples[windows_done:]
        if remaining:
            _, transform_test = build_img_transforms(cfg)
            try:
                _, seq_transform_test = build_seq_transforms(cfg)
            except Exception:
                seq_transform_test = None
            ds = ImageDataset(cfg, remaining, input_wh, output_wh,
                              transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
            # Streaming forces single-process loading: the in-memory frame store + the
            # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
            loader_workers = 0 if streaming else num_workers
            loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
            detector = build_detector(cfg)

            from contextlib import ExitStack

            t0 = time.time()
            done = windows_done
            win_idx = windows_done
            log_every = max(1, log_every_batches)
            flush_n = max(1, flush_every)
            pending = []
            logger.info(f"detecting on {len(remaining)} remaining windows "
                        f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
                        f"source={'video-stream' if streaming else 'frames-dir'})...")
            det_f = open(det_jsonl, "a")
            try:
                with ExitStack() as stack:
                    stack.enter_context(torch.no_grad())
                    # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
                    # in-memory decoded frame for that synthetic path; every other line of
                    # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
                    # The detector's first needed frame is `windows_done` (window i starts
at
                    # frame i), so prime the store there for a resume.
                    if streaming:
                        stack.enter_context(
                            _streaming_read_image_patch(frame_loader, len(frames),
                                                        window=frames_in,
start_index=windows_done))
                    for bi, batch in enumerate(loader):
                        imgs, _hms, trans = batch[0], batch[1], batch[2]
                        img_paths = [list(t) for t in batch[-1]]   # [frame_in][batch] ->
path
                        batch_results, _ = detector.run_tensor(imgs, trans)
                        nb = imgs.shape[0]
                        for ib in range(nb):
                            frames_payload = []
                            for ie in sorted(batch_results[ib].keys()):
```

```
                                p = img_paths[ie][ib]
                                fid = _frame_id(p)
                                plain = [_det_plain(d) for d in batch_results[ib][ie]]
                                frames_payload.append([fid, plain])
                                det_by_fid[fid].extend(plain)
                            pending.append({"w": win_idx, "f": frames_payload})
                            win_idx += 1
                        done += nb
                        if len(pending) >= flush_n or done >= windows_total:
                            _append_windows(det_f, pending)
                            pending = []
                            base_manifest["windows_done"] = done
                            write_manifest(cache_dir, base_manifest)
                        if (bi + 1) % log_every == 0 or done >= windows_total:
                            el = time.time() - t0
                            new_done = done - windows_done
                            rate = new_done / el if el > 0 else 0.0
                            eta = (windows_total - done) / rate if rate > 0 else 0.0
                            logger.info(f"{done}/{windows_total} "
                                        f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
                                        f"elapsed {el:.0f}s | ETA {eta:.0f}s")
                    if pending:
                        _append_windows(det_f, pending)
                        base_manifest["windows_done"] = done
                        write_manifest(cache_dir, base_manifest)
            finally:
                det_f.close()
            logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
        else:
            logger.info("detector cache already complete -> tracker-only re-run")

        # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
        tracker = build_tracker(cfg)
        tracker.refresh()
        rows = []
        for fid in sorted(det_by_fid.keys()):
            r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
            rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))

        _atomic_write_trajectory(out_csv, rows)
        visible = sum(1 for _, v, *_ in rows if v)
        logger.info(f"wrote {len(rows)} rows -> {out_csv}")
        logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
        return out_csv


    def extract_frames(video_path: str, frames_dir: str) -> str:
        """Decode a video to PNG frames named by source frame index (00000000.png ...).

        Idempotent: if the dir already holds the expected number of frames (matching the
        container's reported frame count) we skip re-decoding ? re-extraction after a
        reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
        extracting with ffmpeg on the host and passing --frames_dir.
        """
        import cv2
        os.makedirs(frames_dir, exist_ok=True)
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        existing = len(glob.glob(osp.join(frames_dir, "*.png")))
        if expected > 0 and existing >= expected:
            cap.release()
            logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
```

```
            return frames_dir
        i = 0
        while True:
            ok, frame = cap.read()
            if not ok:
                break
            cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
            i += 1
        cap.release()
        logger.info(f"extracted {i} frames -> {frames_dir}")
        return frames_dir


    def _probe_frame_count(video_path: str) -> int:
        """Container-reported frame count via cv2 (0 if unknown). Used to size the
stream."""
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        cap.release()
        return n


    def make_video_frame_loader(video_path: str):
        """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.

        ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
        long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
        detector's sliding-window order); it reads sequentially and only uses a frame-seek
to
        skip forward when a *resume* starts past the current position. Each ``cap.read()``
        returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
would
        yield (PNG is lossless), so the detector sees identical pixels to the disk path.

        Returns the count hint from the container so the caller can build the frame list;
        ``release`` closes the capture.
        """
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            raise SystemExit(f"cannot open video: {video_path}")
        count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
        state = {"pos": 0}  # next index cap.read() will return

        def frame_loader(index: int):
            index = int(index)
            if index < state["pos"]:
                raise RuntimeError(
                    f"streaming decode is forward-only; got index {index} after
{state['pos']}")
            if index > state["pos"]:
                # Forward skip only happens when resuming past cached windows. Seek once.
                cap.set(cv2.CAP_PROP_POS_FRAMES, index)
                state["pos"] = index
            ok, frame = cap.read()
            if not ok:
                raise RuntimeError(f"failed to decode frame {index} from {video_path}")
            state["pos"] += 1
            return frame

        def release():
            cap.release()
```

```
626
627         return frame_loader, count_hint, release
628
629
630     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
631                             limit: int = 0, batch_size: int = 8,
632                             log_every_batches: int = 50, cache_dir: "str | None" = None,
633                             flush_every: int = 1000, fresh: bool = False) -> str:
634         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
635         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
636         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
637         ``run``'s docstring for why.
638         """
639         frame_loader, count_hint, release = make_video_frame_loader(video_path)
640         try:
641             if count_hint <= 0:
642                 raise SystemExit(
643                     f"could not determine frame count for {video_path}; cannot stream "
644                     f"(fall back to frame extraction with --frames_dir)")
645             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
646             source_key = "video:" + osp.abspath(video_path)
647             return run(frame_list, weights, out_csv, sport, limit,
648                        batch_size=batch_size, num_workers=0,
649                        log_every_batches=log_every_batches, cache_dir=cache_dir,
650                        flush_every=flush_every, fresh=fresh,
651                        frame_loader=frame_loader, source_key=source_key)
652         finally:
653             release()
654
655
656     def main():
657         ap = argparse.ArgumentParser()
658         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
659         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
660         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
661         ap.add_argument("--stream-video", action="store_true",
662                         help="FAST PATH: decode --video on the fly and feed frames straight
to "
663                              "the detector (no PNG extraction). Output-equivalent to the
disk path.")
664         ap.add_argument("--weights", required=True)
665         ap.add_argument("--out", required=True)
666         ap.add_argument("--sport", default="badminton")
667         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
668         ap.add_argument("--batch-size", type=int, default=8)
669         ap.add_argument("--num-workers", type=int, default=4)
670         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671         ap.add_argument("--cache-dir", default=None,
672                         help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
673         ap.add_argument("--flush-every", type=int, default=1000,
674                         help="fsync the detector cache every N windows (bounds reboot
loss)")
675         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676         args = ap.parse_args()
677
678         if args.stream_video:
679             if not args.video:
680                 raise SystemExit("--stream-video requires --video")
```

```
681             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
682                                 batch_size=args.batch_size,
683                                 log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
684                                 flush_every=args.flush_every, fresh=args.fresh)
685             return
686
687         frames_dir = args.frames_dir
688         if args.video:
689             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
690         if not frames_dir:
691             raise SystemExit("provide --frames_dir or --video")
692         run(frames_dir, args.weights, args.out, args.sport, args.limit,
693             batch_size=args.batch_size, num_workers=args.num_workers,
694             log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
695             flush_every=args.flush_every, fresh=args.fresh)
696
697
698     if __name__ == "__main__":
699         main()
700
```

**4. assistant (2026-06-19T16:57:14.582Z)**

All three sub-claims map cleanly onto the code. Let me confirm the key cross-module references inside this file (the detector candidate dict shape and the tracker collapse) are all internally consistent.

The evidence is fully contained in the file I read. Let me record the verdict.

**5. user (2026-06-19T16:57:28.266Z)**

Structured output provided successfully

**6. assistant (2026-06-19T16:57:34.389Z)**

Verified all three sub-claims against `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\wasb_infer.py`. All VERIFIED, with the caveat (noted in each claim's evidence) that the literal construction of the candidate dicts and the tracker's collapse algorithm live in WASB's imported `detectors`/`trackers` packages, not in this file ? but the contracts are unambiguously evidenced at the call sites here:

- (a) per-frame LIST of `{xy, score, scale}` candidates ? lines 101-104, 503, 107-112
- (b) `det_by_fid` accumulates ALL candidates via `.extend` (never overwrite) ? lines 439/163, 505, 179
- (c) `tracker.update` collapses the list to ONE `(visi, x, y)` per frame ? lines 533-538